

UE23CS352B: SOFTWARE ENGINEERING LAB — LAB 3

Architectural Pattern Selection & Written Justification

Student Name: Krrish	SRN: PES1UG24CS708
Assigned Scenario: Self-Service Coffee Kiosk	Architecture Selected: Layered (3-Tier)

Architecture Selection:

"We chose Layered Architecture (3-Tier Architecture) for the Self-Service Coffee Kiosk System."

1. Architectural Choice:

We selected the Layered Architecture pattern, structured horizontally into three distinct tiers: **Presentation Layer** (User Interface Component), **Business Layer** (Order Manager, Payment Service, and Receipt Printer Components), and **Data Layer** (Database Component). Each layer exhibits strong internal cohesion and loose coupling through explicit ball-and-socket interfaces.

2. Two Scenario-Related Reasons for Choice:

- **Hardware Abstraction & Driver Decoupling:** The coffee kiosk terminal interfaces with physical peripherals (touchscreen input and a thermal receipt printer). By encapsulating the receipt printing hardware logic inside a dedicated Receipt Printer Component in the Business Layer, device communication protocols (e.g., ESC/POS driver commands) remain completely isolated. Upgrading printer hardware or paper sensors requires zero modifications to the UI or Database layers.
- **Optimal Fit for Standalone Interactive Terminals:** The kiosk operates as an autonomous, co-located physical station in a busy café. A distributed microservices architecture would introduce needless operational complexity, container overhead, and network serialization costs with no tangible advantage. A layered monolith provides a modular, lightweight executable that delivers rock-solid reliability on embedded kiosk hardware.

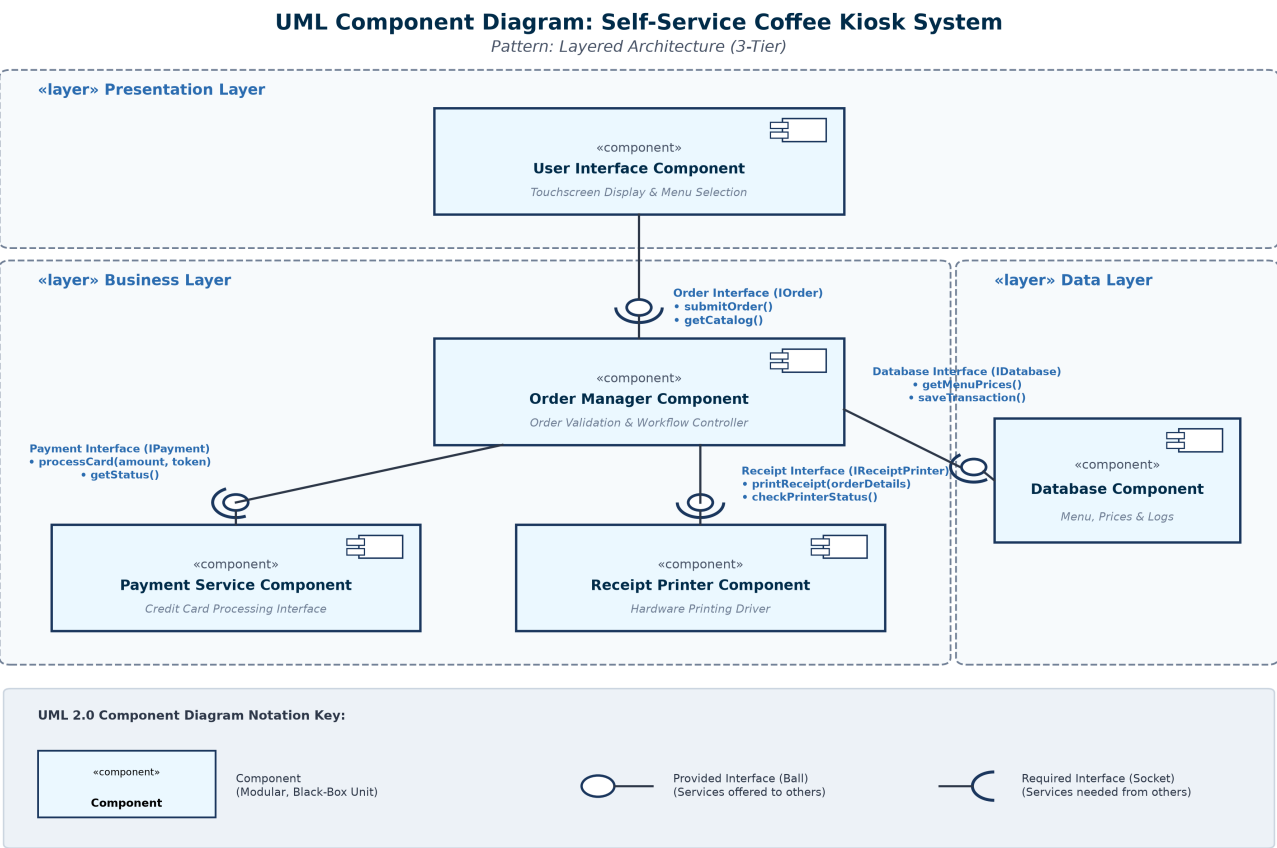
3. Security Advantage (PCI-DSS Cardholder Data Isolation):

Under PCI-DSS security compliance rules, cardholder data and credit card tokens must be strictly isolated from the rest of the application. In our layered architecture, payment processing is entirely encapsulated within the Payment Service Component. The User Interface captures payment details through a dedicated secure widget and immediately forwards tokens across the IPayment interface without persisting them. Credit card numbers are never accessible to the Order Manager or written to the local Database Component, completely protecting sensitive customer data from physical memory inspection.

4. Performance Benefit (Sub-Millisecond In-Memory Execution):

In high-volume peak café operating hours, user interface fluidity and swift checkout transactions are critical to prevent customer queue build-ups. Unlike client-server or microservices patterns where every drink selection, size toggle, or price recalculation incurs network round-trip latency, all components in our layered architecture execute in-process via direct memory calls. Menu retrieval, drink customization, and receipt generation complete in sub-milliseconds, guaranteeing an instantaneous and fluid touchscreen experience.

UML Component Diagram: Self-Service Coffee Kiosk System



Component Name	Layer	Interface Provided/Required
User Interface Component	Presentation Layer	Requires: Order Interface (IOrder)
Order Manager Component	Business Layer	Provides: IOrder Requires: IPayment, IReceiptPrinter, IDatabase
Payment Service Component	Business Layer	Provides: Payment Interface (IPayment)
Receipt Printer Component	Business Layer	Provides: Receipt Interface (IReceiptPrinter)
Database Component	Data Layer	Provides: Database Interface (IDatabase)